NVIDIA-POWERED AI PLATFORM

Call Analysis

Contact Center Intelligence

End-to-end AI platform: Ingest → ASR → Diarization → PII Scrub → Multi-Agent Analysis → Dashboard

6	55+	30+	3
AI Agents	Automated Tests	REST Endpoints	Deployment Targets

Project Proposal

Version 0.4.0 · Python 3.11 · FastAPI · NVIDIA NIM

Built on the NVIDIA free AI stack (hosted NIM via `https://build.nvidia.com`)

August 17, 2026

Contents

1. Executive Summary

Call Analysis is an end-to-end contact center intelligence platform that turns raw call recordings into structured, actionable insight. It combines automatic speech recognition (ASR), multi-speaker diarization, privacy-first PII redaction, and a orchestrated team of specialized large-language-model (LLM) agents – all powered by the **NVIDIA free AI stack** (hosted NIM, Llama-3.1-8B-Instruct) – and presents the results in an NVIDIA-themed interactive dashboard.

The platform is designed for contact center operators, QA teams, compliance officers, and supervisors who need to answer questions such as:

- How well did the agent handle this call, and what coaching would help?
- Did the conversation comply with policy and privacy requirements?
- What was the customer's sentiment and the root cause of the contact?
- What follow-up actions should be pushed to the CRM?

Key Highlights

- **6 specialized AI agents** – QA scorecard, compliance & risk, sentiment & emotion, root cause & intent, CRM action items, and a RAG-based call copilot.
- **Multi-source ingestion** – upload recordings, or bulk-import from local folders, HuggingFace dataset repos, and Kaggle datasets.
- **Privacy by design** – PII detection & redaction (phones, emails, cards, SSN, Aadhaar, account numbers) before any LLM processing.
- **Production-grade core** – FastAPI, PostgreSQL, Redis/Celery, JWT + API keys with RBAC, OpenTelemetry, Prometheus, Grafana, Docker Compose.
- **Three delivery targets** – Docker stack, Electron Windows desktop app, and a standalone PyInstaller/NSIS installer.

2. Problem Statement

Contact centers generate enormous volumes of recorded conversation data, yet most of it is never analyzed. Organizations face several recurring challenges:

1. **No systematic quality assurance.** Agent performance is judged by sparse, manual call sampling rather than continuous, consistent scoring.
2. **Compliance exposure.** Policy violations, privacy leaks, and risky language are discovered after the fact – or not at all.
3. **Siloed insight.** Transcripts, sentiment, and action items live in separate tools and never feed back into coaching or CRM workflows.
4. **High cost of AI adoption.** Many speech-to-insight solutions require expensive GPU infrastructure and proprietary cloud pricing.
5. **Privacy risk.** Sending raw recordings containing personal data to LLM services without scrubbing is a compliance hazard (GDPR, HIPAA, local regulations).

The goal of this project is to deliver a **complete, self-contained, low-cost** platform that solves these problems on the NVIDIA free AI stack, deployable from a single laptop up to a containerized production stack.

3. Solution Overview

One-line pipeline: *Ingest → ASR (Whisper) → multi-speaker diarization → PII scrub → multi-agent analysis → NVIDIA-themed dashboard.*

The platform follows a modular, layered design. Each stage is an independent component with a well-defined interface, which keeps the system testable and lets individual stages be swapped (for example, replacing the heuristic diarizer with pyannote.audio, or switching ASR model sizes without touching the rest of the pipeline).

Design principles

- **Free AI stack first** – hosted NVIDIA NIM for LLMs; faster-whisper for ASR with GPU → CPU fallback.
- **Privacy before intelligence** – PII is redacted before any LLM call.
- **Local-first, cloud-ready** – one command on a laptop; Docker Compose for production; native Windows desktop distribution.
- **Observable by default** – structured logging, OpenTelemetry tracing, Prometheus metrics, and a bundled Grafana dashboard.

4. System Architecture

Figure 1 shows the layered architecture. The **web dashboard** and **Electron desktop shell** are the two presentation front-ends. Both talk to a single **FastAPI** service that owns the domain logic: ingestion, the analysis pipeline, agent orchestration, and the file store. Background processing is decoupled through Celery + Redis, durable state lives in PostgreSQL, and every layer emits telemetry.

4.1 Component Responsibilities

Component	Responsibility
FastAPI service	REST API, validation (Pydantic v2), uploads, static dashboard, CORS, health checks
Pipeline runner	End-to-end job orchestration: audio → report with progress & recovery
Celery + Redis	Async job queue, retries, beat scheduling, circuit breakers, job status tracking
Agent orchestrator	Fans a call transcript out to 6 agents; aggregates JSON scorecards
CallStore	Lightweight JSON persistence of calls, analyses, and metadata (no DB required for demo)
PostgreSQL / Alembic	Multi-tenant relational persistence, users, orgs, API keys, migrations
Telemetry stack	OTel traces, Prometheus metrics, Grafana dashboards, JSON logging

Table 1: Primary components and their responsibilities.

5. Core Processing Pipeline

Each uploaded recording flows through the pipeline shown in Figure 2. Stages are chained, and every stage records progress so long jobs can be resumed after a restart (`recover_interrupted_jobs`).

5.1 Stage Details

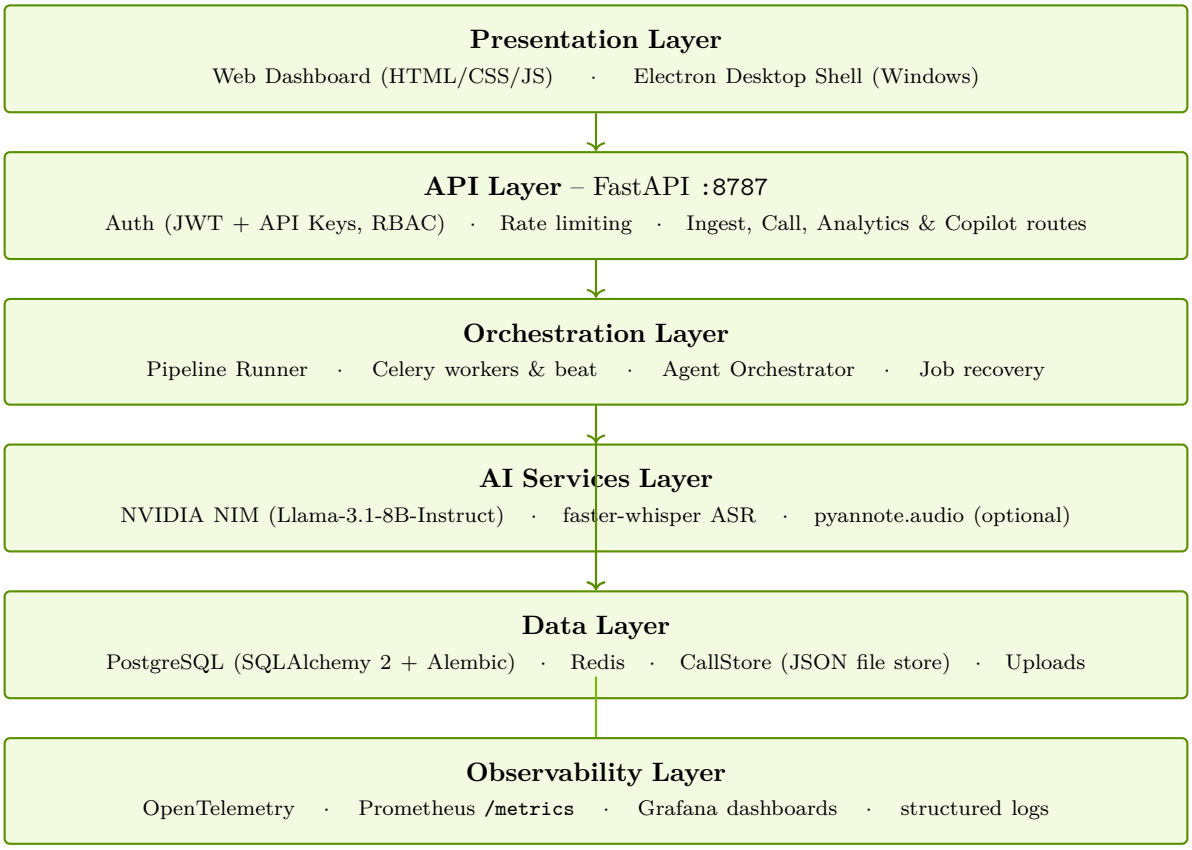

Figure 1: Layered system architecture of the Call Analysis platform.

Stage	Details
Ingest	Multi-file upload; recursive folder scan; HuggingFace dataset repos (public tree API); Kaggle dataset archives (zip-slip-safe extraction, creds via env or `~/.kaggle/kaggle.json`); optional auto-analysis and dedupe
Preprocess	ffmpeg-based conversion/normalization so downstream stages see consistent audio
ASR	faster-whisper with configurable model size; GPU acceleration with automatic CPU fallback
Diarization	Heuristic speaker turn-taking for 2–4 speakers; optional pyannote.audio integration
PII scrub	Regex + ML detection; redaction of emails, phones, credit cards, SSN, Aadhaar, account numbers before LLM calls
Agents	Parallel execution of 6 agents against the scrubbed transcript; structured JSON outputs
Report	Aggregated scorecard, sentiment timeline, compliance flags, root cause, CRM actions, copilot index

Table 2: Pipeline stages and implementation details.

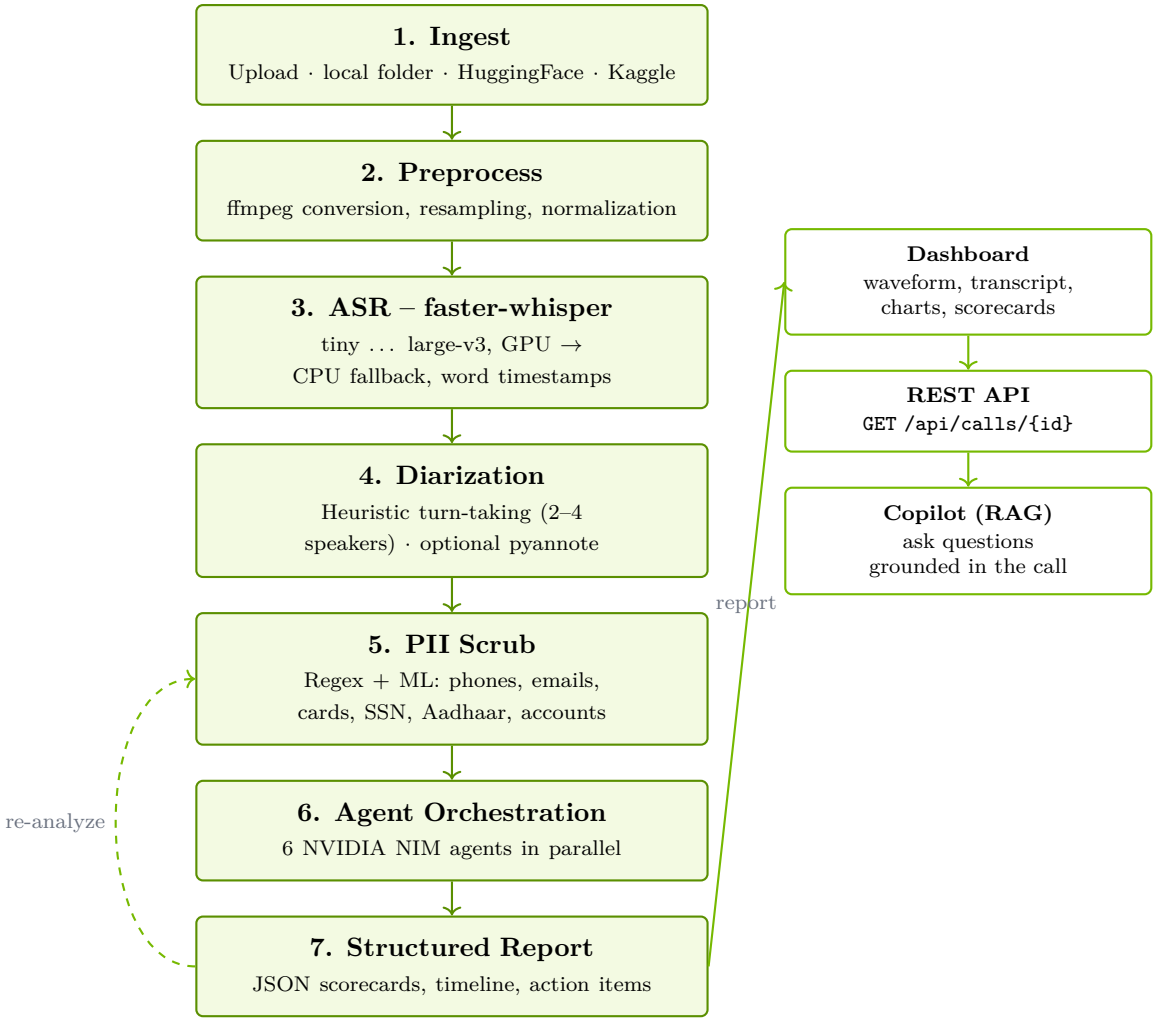

Figure 2: End-to-end processing pipeline for a single call recording.

6. AI Agent Framework

The heart of the analysis is an **agent orchestrator** that runs six specialized LLM agents against the scrubbed transcript (Figure 3). All agents call the same hosted NVIDIA NIM endpoint (`Llama-3.1-8B-Instruct`) with dedicated system prompts and JSON output contracts, so results are machine-readable and consistent.

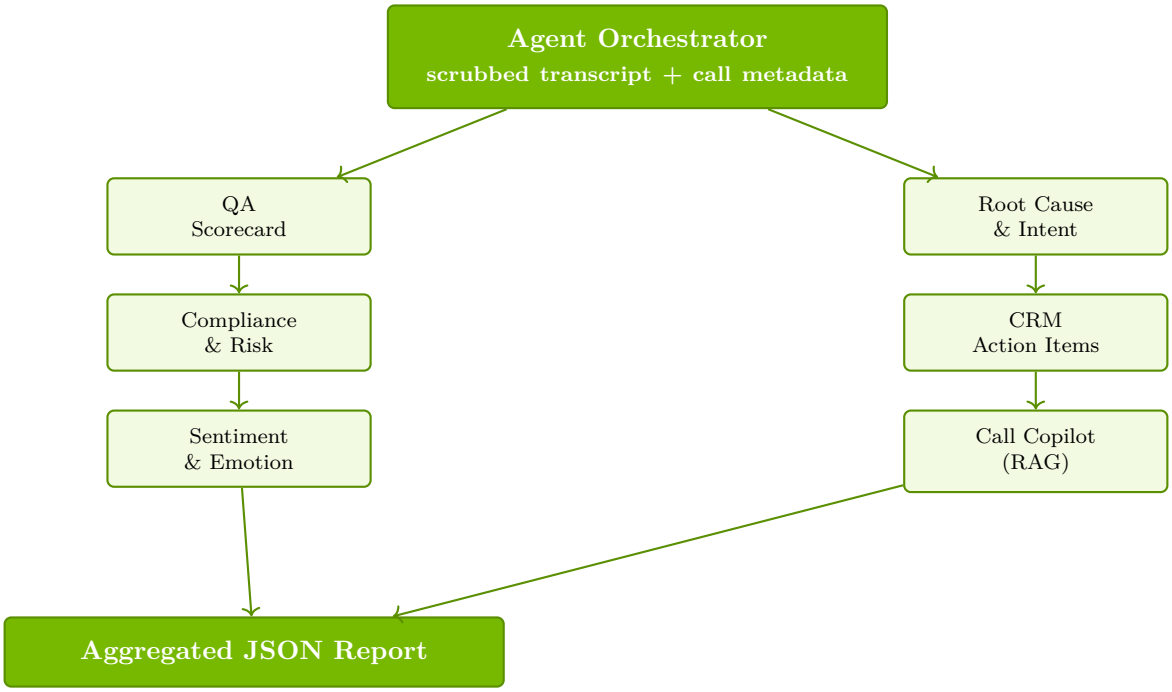

Figure 3: Multi-agent orchestration: one transcript, six specialized agents, one aggregated report.

Agent	Output
QA Scorecard	0–100 performance score, structured criteria, coaching tips
Compliance & Risk	Policy/privacy violations with severity levels and evidence spans
Sentiment & Emotion	Per-turn emotional vectors and a timeline chart
Root Cause & Intent	Why the customer called; resolution status tracking
CRM Action Items	Prioritized follow-ups ready to push to CRM systems
Call Copilot (RAG)	Supervised Q&A grounded in the call transcript

Table 3: The six AI agents and their structured outputs.

7. Audio Ingestion & Dataset Support

Beyond single-file upload, the platform supports **bulk ingestion** from three additional sources, making it easy to load real or synthetic call corpora for demos, benchmarks, and pilot deployments:

Supported formats: `.m4a`, `.wav`, `.mp3`, `.ogg`, `.flac`, `.webm`, `.aac`, `.opus`. All network access uses `httpx` (already a project dependency), so no extra packages are required.

8. Technology Stack

9. REST API Overview

The API is grouped by domain; the full surface is self-documented at `/docs` (Swagger UI).

Source	How it works
Local folder	`POST /api/calls/import-folder` – recursive scan for audio files, dedupe by filename, optional auto-analysis
HuggingFace	`POST /api/calls/import-hf` – public repo-tree API; no credentials required
Kaggle	`POST /api/calls/import-kaggle` – archive download with zip-slip-safe extraction; credentials from env or `~/.kaggle/kaggle.json`

Table 4: Bulk ingestion sources (all also available from the dashboard *Ingest* panel).

Area	Technologies
Web framework	Python 3.11, FastAPI, Uvicorn, Pydantic v2, pydantic-settings, httpx
Database & ORM	PostgreSQL, SQLAlchemy 2, Alembic, asyncpg, psycopg
Task queue	Celery, Redis, kombu
Auth & security	python-jose (JWT), passlib/bcrypt, API keys, slowapi rate limiting
Audio & AI	faster-whisper, librosa, numpy, NVIDIA NIM (Llama-3.1-8B-Instruct), pyannote (optional)
Observability	OpenTelemetry, Prometheus, Grafana, structlog, python-json-logger
Packaging	Docker + Compose, PyInstaller, NSIS, Electron (desktop shell)
Testing	pytest, pytest-cov, FastAPI TestClient, ruff, mypy

Table 5: Technology stack by area.

Endpoint		Purpose
`POST /api/auth/login`, `/api/auth/me`	`GET`	JWT authentication
`/api/organizations`		Multi-tenant organization CRUD (admin)
`/api/users`		User management (admin)
`/api/api-keys`		API key lifecycle (analyst+)
`GET /api/analytics`		Fleet-level analytics
`GET /api/calls`, `/api/calls/{id}`	`GET`	List and inspect calls
`POST /api/calls/upload`		Upload one or more recordings
`POST /api/calls/import-folder | import-hf | import-kaggle`		Bulk ingestion
`POST /api/calls/{id}/analyze`		Trigger / re-run analysis
`POST /api/calls/{id}/copilot`		RAG copilot question
`DELETE /api/calls/{id}`		Delete a call
`/api/webhooks`		Outbound webhook management (admin)

Table 6: Primary REST endpoints.

10. Security & Compliance

- **Authentication** – JWT bearer tokens and API keys; passwords hashed with bcrypt.
- **Authorization (RBAC)** – `Admin` / `Analyst` / `Viewer` roles; organization-scoped data isolation (multi-tenancy).
- **Rate limiting** – per-client limits via slowapi to prevent abuse.
- **PII redaction** – regex + ML detection scrubs personal data *before* any LLM call; redacted transcripts are what the agents see.
- **Safe ingestion** – Kaggle archives are extracted with zip-slip protection; dataset identifiers are validated against a strict pattern.
- **Hardening** – CORS controls, security headers, input validation (Pydantic), structured audit events, and HTTPS guidance for production.

11. Observability & Monitoring

The platform is observable by default:

- **Metrics** (Prometheus, `/metrics`): `calls_processed_total`, `call_processing_duration_seconds`, `api_requests_total`, `api_request_duration_seconds`, `active_jobs`, `queue_depth`, and system metrics (`system_cpu_percent`, `system_memory_percent`).
- **Tracing** – OpenTelemetry instrumentation for FastAPI, SQLAlchemy, Redis, and httpx; OTLP exporter; bundled OpenTelemetry Collector configuration.
- **Dashboards** – a Grafana dashboard ships with the repository (`grafana/dashboards/call-analysis-platform.json`).
- **Logging** – structured JSON logs via structlog / python-json-logger.

12. Deployment Options

12.1 Docker Compose (production stack)

A single `docker-compose up -d` brings up PostgreSQL, Redis, the API (4 workers), two Celery workers, Celery beat, an OpenTelemetry Collector, Prometheus, and Grafana.

12.2 Electron Windows Desktop App

An Electron shell spawns the Python backend as a child process, waits for `/api/health`, and loads the dashboard in a native window. Data lives under `%APPDATA%\Call Analysis\data`. A one-command build pipeline (`build_electron.py`) produces an NSIS installer (`CallAnalysis-Setup-<version>.exe`).

12.3 Standalone Executable

`build_windows.py` packages the entire backend into a single PyInstaller EXE (`CallAnalysis.exe`) with an optional NSIS installer, for air-gapped or non-technical deployments.

Developer experience – `python -m call_analysis.serve` starts the full dashboard on `http://127.0.0.1:8787` with no external services; the JSON `CallStore` makes the demo path run on a single machine.

13. Testing & Quality

- **Unit tests** – 55+ tests covering API endpoints (including the ingestion suite), configuration, storage/models, diarization, PII redaction, and the NIM client (mocked).
- **Integration tests** – marked `integration` for environments with live services (PostgreSQL, Redis, ffmpeg).
- **Static analysis** – ruff (lint + import sorting) and mypy (type checking) enforced in CI-style workflow.
- **Mock-based network tests** – HuggingFace and Kaggle flows are tested with `httpx.MockTransport`, so the suite is deterministic and offline.

14. Roadmap & Future Work

1. **Production hardening** – wire the Celery worker to the live API path (Postgres-backed jobs) and complete the multi-tenant auth flow end-to-end.
2. **More model options** – drop-in support for additional NVIDIA NIM models and open-weight ASR models.
3. **CRM integrations** – first-class connectors (Salesforce, HubSpot) for action-item sync.
4. **Real-time mode** – streaming transcription for live-call assist.
5. **Scale-out** – object storage (S3/GCS) for uploads and horizontal worker autoscaling.
6. **Localization** – multilingual scorecards and compliance rules.

15. Conclusion

Call Analysis delivers a complete, NVIDIA-powered contact center intelligence platform: it ingests calls from files, folders, and public datasets; transcribes and diarizes them; scrubs sensitive data; analyzes them with six specialized AI agents; and presents everything in an interactive, NVIDIA-themed dashboard. It is testable, observable, and ships in three deployment shapes – Docker, a native Windows desktop app, and a standalone executable – all built on the NVIDIA free AI stack.

Proposal prepared by the Call Analysis team. For a live walkthrough, run `python -m call_analysis.serve` and open `http://127.0.0.1:8787`.